Decision Brief: Scaling Remote Farmer

Date: 2026-03-12 | **Audience**: Solo developer, team leads | **Phase**: Compiled

Executive Summary

Remote Farmer should scale via three parallel workstreams -- all prototyped and validated: **multi-session** (session-keyed Maps, tested [p003]; GC + caps [p012]; full hardening [p014]), **multi-sprint** (sprints/ directory with git-derived detection, tested [p004, p013]; compiler integration [p015]), and **repo cartography** (compiler-generated topic-map manifest in <10ms, tested [p001, p007]). Security hardening (cookie auth, CORS restriction, Referrer-Policy) is implemented [p005], backwards compatibility is verified with 10/10 tests passing [p006].

Fifteen prototypes have landed across the sprint. All seven multi-session challenge risks (x010--x016) are now resolved: x010/x011 by session GC [p012] and x012--x016 by session hardening [p014]. All six WebSocket risks (x004--x009) were resolved by WebSocket hardening [p011]. Stakeholder feedback [f001] on config drift is validated by git-derived sprint detection [p013], now integrated into the compiler [p015].

The sprint is feature-complete. Remaining work is operational: named Cloudflare tunnel and integration wiring.

Recommendation

The sprint is substantially complete. Thirteen of fifteen roadmap items are DONE. All challenge-identified risks are resolved. Remaining work:

1. **Named Cloudflare tunnel** -- replace quick tunnel for stable subdomain and native SSE support [r027, r036]. WebSocket fallback [p009] reduces urgency but named tunnel remains best practice. Estimated 2 hours.
2. **Sprint detection wiring** -- integrate detect-sprints.js [p013, p015] as the server's canonical sprint resolver (replace `--sprints-dir` hot-reload with git-aware detection). Estimated 0.5 hours.

Total estimated remaining effort: 2.5 hours.

Evidence Summary

Multi-Session (22 claims after compiler resolution, tested)

Hook payloads include stable `session_id` UUID as a natural routing key [r001]. SessionStart/SessionEnd lifecycle hooks provide explicit session boundaries [r002]. Architecture uses single server with `Map<session_id, SessionState>` and lazy `getSession(id)` initializer [r003, r004]. Dashboard shows unified feed with session color-coding via hue-from-hash [r005, p002]. Heartbeat timeout (5min) handles crashed sessions [r007]. Estimated 2--4 hours effort [r008]. **Prototype working** -- SessionState class, session-keyed Maps, SSE tagging, pill-bar switcher all implemented [p003].

Key constraint: Lifecycle hooks are command-only, not HTTP -- requires curl wrapper [w002]. SessionStart fires on resume/clear/compact, not just new sessions [w001].

Risk: Concurrent permission requests from multiple sessions create approval confusion -- mitigated with prominent session labels + colors [r006].

Session GC implemented [p012]:

- MAX_SESSIONS guard (default 50) on getSession() -- evicts oldest ended/stale session when cap reached, rejects with 503 if all active
- Periodic reaper (every 60s) prunes ended sessions after 5 min TTL and stale sessions after 30 min TTL
- Broadcasts 'session_removed' event so dashboard UI stays in sync
- Resolves x010 (memory leak) and x011 (no session cap)

Session hardening implemented [p014]:

- Source fingerprint prevents session ID spoofing -- rejects requests from different source [x012 resolved]
- Missing session_id generates unique derived ID instead of merging into 'default' [x013 resolved]
- Out-of-order lifecycle events buffered -- session_end before session_start queued and replayed (30s TTL) [x014 resolved]
- State persistence uses dirty flag + 2s debounce to prevent write races; force-save on shutdown [x015 resolved]
- Dashboard session bar shows max 5 sessions with expand/collapse, priority sorting, count indicator [x016 resolved]

Multi-Sprint (9 claims, tested)

Each sprint lives in `sprints/<slug>/` with own claims.json. Dashboard shows collapsible sprint card list [r022]. Server adds `--sprints-dir` with hot-reload via `fs.watchFile` [r021]. Cross-sprint references use `sprint-slug:claim-id`

format [r024]. Compiler resolves cross-sprint references without schema changes [r023]. Estimated 3--5 hours effort [r026]. **Prototype working** -- sprint scanning, `/api/sprints` endpoint, hot-reload, backwards compatible with `--claims` flag [p004].

Git-derived sprint detection [p013]: detect-sprints.js determines the active sprint purely from filesystem + git -- no config pointer needed. Scans for claims.json files, reads meta.phase for archived status, queries git log for last commit date and commit count, then ranks candidates. Measured at ~150ms for 3 sprints. **Validates f001**: config should not duplicate git-derivable state.

Compiler integration [p015]: wheat-compiler.js now runs detect-sprints.js on every compilation, adding a 'sprints' array to compilation.json with summary entries for all detected sprints. Active sprint compiled fully; others get summary entries. generate-manifest.js delegates to detect-sprints.js for git-aware sprint detection. Non-fatal on failure.

Feedback: Config should not duplicate git-derivable state [f001] -- validated by p013 and p015. The active sprint is inferred from git recency. Supersedes r020 and r025.

Repo Cartography (10 claims, tested)

`wheat-manifest.json` is a topic map (not file tree) -- AI searches by concept [r017]. Auto-generated by compiler so it never drifts from source [r011]. Single `Read` call replaces 3--5 Glob/Grep calls [r012]. CLAUDE.md stays behavioral, manifest is structural [r015]. **Generator built AND wired into compiler** -- produces manifest in <10ms (~6.7ms measured), ~7.2KB with sprint detection, non-fatal on failure [p001, p007, p015].

Urgency: Repo already at 157 files (not the estimated 80) [x001]. The `examples/` directory creates 62 duplicate-named files causing search noise [x002]. Context window pollution -- not I/O -- is the real cost [x003].

Risk: Manifest can mislead if out of date -- compiler-generated approach mitigates but does not eliminate [r014]. Search tools still fall back to Glob/Grep if manifest is missing [r013].

Security (16 claims, tested)

Cloudflare quick tunnels do not support SSE -- dashboard falls back to 2s polling [r027]. Quick tunnels are dev-only with 200 concurrent request limit [r028]. Token-in-URL leaks through browser history, Referer headers, proxy logs [r029]. CORS wildcard allows any website to make authenticated API calls [r030]. Full compromise on token leak: approve arbitrary tool executions, set autonomous mode, inject responses [r031]. The `/api/ask` injection vector allows

attackers to respond to pending questions before the real user [r032]. Token brute force is infeasible (2^128 entropy, timingSafeEqual) but no rate limiting exists [r033]. No token rotation -- only fix for a leak is full restart [r034]. Token embedded in served HTML via template replacement [r037].

Mitigations implemented [p005]: Cookie-based auth (HttpOnly, SameSite=Strict) with 302-redirect to clean URL, CORS restricted to request origin, Referrer-Policy: no-referrer, X-Content-Type-Options: nosniff.

WebSocket transport [p009]: RFC 6455 handshake and frame encoding (~120 LOC, zero deps). Dashboard cascade fallback: WebSocket first (works through CF quick tunnels), SSE second (works local/direct), 2s polling baseline always active. 30s ping/pong heartbeat. Verified: 101 upgrade with valid token, 401 rejection on invalid.

WebSocket hardening [p011]: Patched 6 risks from /challenge p009 -- Origin header validation (x004), token as first WS message instead of URL query (x005), 1MB max buffer with forced disconnect (x006), FIN bit + continuation frame accumulation (x007), RFC 6455 close frame opcode 0x88 (x008), client-side exponential backoff reconnect with SSE fallback (x009). All resolved, zero deps maintained.

CSRF protection + token rotation [p010]: Per-session CSRF tokens (crypto.randomBytes, 24h TTL) delivered via init payloads (SSE, WebSocket) and /api/state polling. All mutating dashboard endpoints validate X-CSRF-Token header, returning 403 on mismatch. Hook endpoints (localhost-only) exempt. Configurable auto-rotation (--token-rotation-interval) and on-demand rotation via POST /api/admin/rotate-token. Retired tokens have configurable grace period (--token-grace-period, default 60s) with timing-safe comparison. Dashboard auto-updates cookie on rotation broadcast. Zero dependencies, ~80 LOC added.

Recommended next steps: Named tunnel for SSE + OAuth [r036].

Compatibility (4 claims, tested)

Zero npm dependencies constraint maintained [d004]. **Backwards compatibility verified** -- 10-test automated suite confirms single-session workflow survives all refactors: token auth, cookie auth, /api/state, hook fallback to 'default' session, SSE stream, login page, auth rejection all working [p006]. WebSocket frame decoder now handles FIN bit and continuation frames per RFC 6455 [x007 resolved]. Close frame handling corrected to opcode 0x88 [x008 resolved].

Performance (6 claims, tested)

Manifest generator runs in <10ms [p001]. Single Read call replaces 3--5 search calls [r012]. Performance scales to 10+ sprints and 500+ claims without degrading [d005]. Current repo at

157 files makes cartography urgent [x001]. Estimated 30% context window reduction via manifest approach [r016].

State Persistence (6 claims, tested)

All server state was held in memory with zero persistence -- restart loses everything [r038]. State classifies into must-persist (trust levels, rules, activity log) vs ephemeral (pending permissions, SSE connections) [r040]. **JSON file persistence now implemented** [p008]: atomic writes (tmp+rename) to `.farmer-state.json`, restores trust levels, session rules, and activity logs on restart. Graceful shutdown via SIGTERM/SIGINT handlers denies pending permissions and flushes state. 30s periodic auto-save. ~45 LOC, zero dependencies. Verified: state survives full stop/start cycle. Event sourcing is over-engineered for this scale (<100KB state, <1/sec writes) [r042].

Persistence races resolved [p014]: State persistence now uses dirty flag + 2s debounce. Force-save on shutdown. Resolves x015.

Reliability (2 claims, resolved)

WebSocket receive buffer grew without bounds [x006] -- now capped at 1MB with forced disconnect. Client-side WebSocket had no reconnection logic [x009] -- now uses exponential backoff (1s--16s, 5 attempts) before SSE fallback. Both resolved by p011.

WebSocket Hardening (1 claim, tested)

All 6 WebSocket risks patched in a single prototype [p011]: Origin validation, token-as-first-message, buffer cap, FIN bit handling, close frame compliance, reconnection with backoff. Zero dependencies maintained.

Tradeoffs and Risks

Risk	Severity	Evidence	Mitigation
Token-in-URL leakage [r029]	High	web	Cookie auth implemented [p005]
CORS wildcard [r030]	High	documented	Restricted to origin [p005]
Full compromise on token leak [r031]	High	documented	Cookie auth + token rotation [p005, p010]
SSE broken through quick tunnels [r027]	High	documented	WebSocket fallback done [p009]; named tunnel recommended

/api/ask injection [r032]	Medium	documented	CSRF tokens implemented [p010]
Approval confusion with concurrent sessions [r006]	Medium	stated	Session labels + colors
No token rotation [r034]	Medium	documented	Rotation implemented [p010]
Session GC missing [x010]	High	documented	RESOLVED by p012
Session count uncapped [x011]	High	documented	RESOLVED by p012
Session ID spoofing [x012]	Medium	documented	RESOLVED by p014
Sessionless conflation [x013]	Medium	documented	RESOLVED by p014
Out-of-order lifecycle [x014]	Medium	documented	RESOLVED by p014
State persistence races [x015]	Low	documented	RESOLVED by p014
Dashboard UX at scale [x016]	Medium	documented	RESOLVED by p014
All state in-memory [r038]	Medium	documented	JSON snapshots implemented [p008]
Manifest can mislead if stale [r014]	Low	documented	Compiler-generated, auto-updates [p007]
Config duplicates derivable state [f001]	Low	stated	VALIDATED by p013, integrated by p015

Resolved Conflicts

f001 vs r020/r025 (Resolved by p013/p015): Stakeholder feedback argued config should not duplicate git-derivable state. detect-sprints.js [p013] validates this, and p015 integrates it into the compiler. Active sprint determined from filesystem + git in ~150ms. r020 and r025 superseded.

x010/x011 (Resolved by p012): Session garbage collection and session count cap implemented. MAX_SESSIONS=50 guard, 60s reaper with TTL-based eviction, 'session_removed' broadcast.

x012--x016 (Resolved by p014): Session hardening fixes all five remaining multi-session risks -- source fingerprinting, derived IDs, lifecycle buffering, debounced persistence, and dashboard

pagination.

x004--x009 (Resolved by p011): All 6 WebSocket risks identified by `/challenge p009` resolved -- Origin validation, token transmission, buffer limits, frame handling, close compliance, and reconnection logic.

Implementation Roadmap

Phase	Workstream	Effort	Status
1	Manifest generator	--	DONE (p001)
2	Multi-session refactor	2-4h	DONE (p003)
3	Multi-sprint support	3-5h	DONE (p004)
4	Security hardening	--	DONE (p005)
5	Backwards compat tests	--	DONE (p006)
6	Compiler manifest integration	1h	DONE (p007)
7	State persistence	1h	DONE (p008)
8	WebSocket transport	--	DONE (p009)
9	Token rotation + CSRF	1h	DONE (p010)
10	WebSocket hardening	--	DONE (p011)
11	Session GC + caps	--	DONE (p012)
12	Sprint detection (git-derived)	--	DONE (p013)
13	Session hardening (x012--x016)	--	DONE (p014)
14	Compiler sprint integration	--	DONE (p015)
15	Named tunnel / stable subdomain	2h	TODO

Appendix: Claim Inventory

ID	Type	Topic	Evidence	Status	Content (abbreviat
d001	constraint	multi-session	stated	active	Must support multip concurrent sessions

					dashboard
d002	constraint	multi-sprint	stated	active	Dashboard must sup viewing/switching multiple sprints
d003	constraint	cartography	stated	active	Repo must be self-describing for AI mo
d004	constraint	compatibility	stated	active	Zero npm deps, backwards compat
d005	constraint	performance	stated	active	AI search must be measurably faster w cartography
r001	factual	multi-session	documented	active	Hook payloads inclu stable session_id UU
r002	factual	multi-session	documented	active	SessionStart/Sessio lifecycle hooks exist
r003	recommendation	multi-session	documented	active	Single-server, sessi keyed Map architect
r004	recommendation	multi-session	documented	active	Refactor globals to p session state
r005	recommendation	multi-session	documented	active	Unified feed with session color-coding
r006	risk	multi-session	stated	active	Concurrent approva confusion
r007	factual	multi-session	documented	active	5min heartbeat time for crashed sessions
r008	estimate	multi-session	stated	active	2--4 hours estimate effort
r010	factual	cartography	documented	active	Claude Code uses Glob/Grep/Read -- n native index
r011	recommendation	cartography	documented	active	Auto-generate mani via compiler

r012	factual	performance	documented	active	Single Read replace -5 search calls
r013	recommendation	cartography	documented	active	Search tools fall bac without manifest
r014	recommendation	cartography	documented	active	Stale manifest can mislead
r015	risk	cartography	documented	active	Per-directory CLAUDE.md maintenance burder
r016	estimate	performance	stated	active	30% context windov reduction
r017	recommendation	cartography	documented	active	Topic map, not file t
r020	recommendation	multi-sprint	documented	superseded	sprints/slug/ directo structure
r021	recommendation	multi-sprint	documented	active	--sprints-dir with hc reload
r022	recommendation	multi-sprint	documented	active	Collapsible sprint ca list UI
r023	recommendation	multi-sprint	documented	active	Compiler resolves cross-sprint referen
r024	recommendation	multi-sprint	documented	active	sprint-slug:claim-id reference format
r025	risk	multi-sprint	documented	superseded	Symlinks break on Windows
r026	estimate	multi-sprint	stated	active	3--5 hours estimate effort
r027	factual	security	documented	active	CF quick tunnels do support SSE
r028	factual	security	documented	active	Quick tunnels are de only, 200 req limit
r029	risk	security	web	active	Token-in-URL leaks Referer/history/logs

r030	risk	security	documented	active	CORS wildcard enab cross-origin attacks
r031	risk	security	documented	active	Full compromise bla radius on token leak
r032	risk	security	documented	active	/api/ask injection via /api/decide
r033	factual	security	documented	active	128-bit token, no rat limiting
r034	risk	security	documented	active	No token rotation, restart = only fix
r035	recommendation	security	web	active	Cookie auth + CSRF CORS restriction
r036	recommendation	security	web	active	Named tunnel or WebSocket transpor
r037	risk	security	documented	active	Token embedded in served HTML
r038	risk	state-persistence	documented	active	All state in-memory, zero persistence
r039	recommendation	state-persistence	documented	active	JSON file snapshots ~40 LOC
r040	factual	state-persistence	documented	active	State classification: must-persist vs ephemeral
r041	recommendation	state-persistence	documented	active	SIGTERM/SIGINT graceful shutdown handlers
r042	estimate	state-persistence	documented	active	Event sourcing over-engineered; JSON snapshots sufficient
w001	factual	multi-session	documented	active	SessionStart fires on resume/clear/compa too

w002	factual	multi-session	documented	active	Lifecycle hooks are command-only, not HTTP
x001	factual	performance	tested	active	Repo at 157 files, no
x002	factual	performance	tested	active	examples/ creates 6 duplicate-named file
x003	risk	performance	documented	active	Context window pollution is real cost
x004	risk	security	tested	resolved	WS upgrade: no Orig validation
x005	risk	security	tested	resolved	WS token in URL qu string
x006	risk	reliability	tested	resolved	WS buffer grows wit bound
x007	risk	compatibility	tested	resolved	WS frame decoder ignores FIN bit
x008	risk	compatibility	tested	resolved	WS close frame viol RFC 6455
x009	risk	reliability	tested	resolved	WS client has no reconnection logic
x010	risk	multi-session	documented	resolved	Abandoned sessions never garbage-colle
x011	risk	multi-session	documented	resolved	No cap on session c
x012	risk	multi-session	documented	resolved	Session ID spoofing allows hijacking
x013	risk	multi-session	documented	resolved	Missing session_id conflates into 'defau
x014	risk	multi-session	documented	resolved	Out-of-order lifecyc causes data loss
x015	risk	multi-session	documented	resolved	State persistence ra under concurrent mutation

x016	risk	multi-session	documented	resolved	Dashboard UX degr at 10+ sessions
p001	factual	cartography	tested	active	Manifest generator: <10ms, ~5KB
p002	recommendation	multi-session	documented	active	Multi-session refact plan
p003	factual	multi-session	tested	active	Multi-session protot working
p004	factual	multi-sprint	tested	active	Multi-sprint prototy working
p005	factual	security	tested	active	Security hardening implemented
p006	factual	compatibility	tested	active	10/10 backwards co tests pass
p007	factual	cartography	tested	active	Manifest wired into compiler, 6.7ms, no fatal
p008	factual	state-persistence	tested	active	JSON persistence + graceful shutdown implemented
p009	factual	security	tested	active	WebSocket transpo with cascade fallbac
p010	factual	security	tested	active	CSRF protection + t rotation implemente
p011	factual	websocket-hardening	tested	active	6 WS risks patched: Origin, buffer, FIN, c reconnect
p012	factual	multi-session	tested	active	Session GC: MAX_SESSIONS=5 reaper, eviction, 503 guards
p013	factual	multi-sprint	tested	active	Git-derived sprint detection: fs+git, ~150ms, validates f

p014	factual	multi-session	tested	active	Session hardening: fingerprint, derived lifecycle buffer, debounce, paginatic
p015	factual	cartography	tested	active	Compiler + manifest sprint detection integration
f001	feedback	multi-sprint	stated	active	Config should not duplicate git-derivat state

---

Compilation certificate: sha256:deca61e6d78d0beb8e5aabe69183a789b7777d9678abf9780ec4784cc187740d | Compiler: wheat v0.2.0 | Claims: 78 (63 active, 2 superseded, 13 resolved) | Compiled: 2026-03-12